

S.S.M.M. COLLEGE, PACHORA

BCA – First Year

EXAM NOTES

CA-121

OBJECT ORIENTED PROGRAMMING IN C++

KBCNMU Affiliated | Units 1–4 | Exam-Ready Notes

UNIT 1: OOPs IN C++

(Marks: 10 | Lectures: 08)

1.1 OOP Features

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects. An object contains data (attributes) and code (methods) that operate on that data. OOP helps in designing software in a modular, reusable, and maintainable way.

1. Object:

An object is a real-world entity with state (attributes) and behavior (methods). Example: A Car object has attributes: color, speed; and methods: start(), brake(). In C++, an object is an instance of a class.

2. Class:

A class is a blueprint or template for creating objects. It defines data members (variables) and member functions (methods). Keyword: class.

```
class Student { public: int roll; void display() { cout << roll; } };
```

3. Data Encapsulation:

Encapsulation is the process of binding data and the functions that operate on it into a single unit (class). It hides the internal implementation. Data is accessed only through member functions.

4. Data Abstraction:

Abstraction means hiding complex implementation details and showing only the essential features. In C++, abstraction is achieved using access specifiers (private, public, protected). Example: When you drive a car, you use the steering but don't know the internal engine mechanism.

5. Inheritance:

Inheritance is the mechanism by which a new class (derived/child class) acquires properties and behaviors of an existing class (base/parent class). It promotes code reuse. Example: class Dog : public Animal { ... };

6. Polymorphism:

Polymorphism means 'many forms'. It allows one interface to be used for different data types. Example: The + operator can add integers, floats, or concatenate strings. Types: Compile-time (function overloading, operator overloading) and Runtime (virtual functions).

7. Message Communication:

Objects communicate with each other by sending messages (calling member functions). Message = Object + Method + Parameters. Example: student1.display(); sends a message to student1 to call display().

8. Delegation:

Delegation refers to passing responsibility to another object or class. It is a form of code reuse where one object uses methods of another.

1.2 Class Specification – Defining Members

Syntax:

```
class ClassName { private: // private members (default) public: // public members  
protected: // protected members };
```

Example:

```
class Rectangle { private: int length, breadth; public: void setValues(int l, int b)  
{ length=l; breadth=b; } int area() { return length * breadth; } }; int main() {  
Rectangle r; r.setValues(5, 3); cout << r.area(); // Output: 15 }
```

1.3 Access Specifiers

Specifier	Access within Class	Access in Derived Class	Access Outside Class
private	Yes	No	No
public	Yes	Yes	Yes
protected	Yes	Yes	No

1.4 Constructors

A **constructor** is a special member function that is automatically called when an object is created. It has the same name as the class, no return type (not even void), and is used to initialize data members.

Types of Constructors:

A) Default Constructor: No parameters. Automatically provided if not defined.

```
class Box { public: Box() { cout << "Object created"; } // Default constructor };
```

B) Parameterized Constructor: Accepts arguments to initialize data members.

```
class Box { int w; public: Box(int x) { w = x; } // Parameterized constructor };
```

C) Copy Constructor: Creates a new object as a copy of an existing object.

```
class Box { int w; public: Box(int x) { w = x; } Box(Box &b;) { w = b.w; } // Copy constructor }; Box b1(10); Box b2(b1); // Calls copy constructor
```

D) Constructor Overloading: Multiple constructors with different parameters.

1.5 Destructor

A **destructor** is a special member function that is called automatically when an object goes out of scope or is deleted. It has the same name as the class prefixed with ~ (tilde). No parameters, no return type. Used to release resources (memory, files).

```
class Box { public: ~Box() { cout << "Object destroyed"; } // Destructor };
```

1.6 Friend Class and Friend Function

Friend Function:

A friend function is not a member of a class but can access its private and protected members. Declared with the friend keyword inside the class.

```
class A { int x; public: A(int v){ x=v; } friend void show(A a); // friend declaration }; void show(A a) { cout << a.x; } // accessing private member
```

Friend Class:

When one class is declared as a friend of another class, all member functions of the friend class can access the private and protected members of the other class.

```
class A { int x; public: A(int v){ x=v; } friend class B; // B is friend of A }; class B { public: void display(A a){ cout << a.x; } // Accessing A's private };
```

UNIT 2: INHERITANCE

(Marks: 15 | Lectures: 07)

2.1 Types of Inheritance

Inheritance is the mechanism by which a derived class acquires properties of a base class. It promotes code reuse and establishes a parent-child relationship.

Single Inheritance: One derived class inherits from one base class.

Example: `class Dog : public Animal { };`

Multiple Inheritance: One derived class inherits from more than one base class.

Example: `class C : public A, public B { };`

Multilevel Inheritance: A class inherits from a derived class (chain).

Example: `A → B → C`

Hierarchical Inheritance: Multiple derived classes inherit from one base class.

Example: `Animal → Dog, Cat, Bird`

Hybrid Inheritance: Combination of two or more types of inheritance.

Example: Mix of multiple and multilevel

2.2 Member Accessibility and Visibility Modes

The **visibility mode** determines how the base class members are accessible in the derived class.

Base Member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	Not inherited	Not inherited	Not inherited

2.3 Virtual Base Class

When multiple inheritance causes the same base class to be inherited more than once (diamond problem), a **virtual base class** ensures only one copy of the base class is inherited.

Example (Diamond Problem):

```
class A { public: int x; }; class B : virtual public A {}; // virtual base class C :  
virtual public A {}; // virtual base class D : public B, public C {}; // D gets only  
one copy of A
```

2.4 Benefits of Inheritance

- Code Reusability: Existing code can be reused without rewriting.
- Extensibility: New features can be added to existing classes easily.
- Reduces Redundancy: Common code written once in base class.
- Supports Polymorphism: Enables runtime behavior through virtual functions.
- Maintainability: Changes in base class reflect in all derived classes.

2.5 Virtual Functions

A **virtual function** is a member function declared with the keyword **virtual** in the base class. It enables **runtime polymorphism**. When a virtual function is called through a base class pointer, the appropriate derived class version is called based on the actual object type.

```
class Animal { public: virtual void sound() { cout << "Generic sound"; } }; class  
Dog : public Animal { public: void sound() { cout << "Bark"; } // overrides virtual
```

```
function }; int main() { Animal *ptr; Dog d; ptr = &d; ptr->sound(); // Output: Bark  
(runtime polymorphism) }
```

2.6 Pure Virtual Function and Abstract Class

A **pure virtual function** is a virtual function with no implementation in the base class. It is declared by assigning 0. Syntax: `virtual void funcName() = 0;`

A class containing at least one pure virtual function is called an **Abstract Class**. Objects of abstract classes CANNOT be created. It serves only as a base class.

```
class Shape { public: virtual float area() = 0; // pure virtual function }; class  
Circle : public Shape { float r; public: Circle(float x){ r=x; } float area(){  
return 3.14*r*r; } // must override }; Shape s; // ERROR: cannot create object of  
abstract class Circle c(5); // OK
```

UNIT 3: POLYMORPHISM AND TEMPLATES

(Marks: 10 | Lectures: 07)

3.1 Concept and Types of Polymorphism

Polymorphism means 'many forms'. It is the ability of a function, operator, or object to behave differently based on context.

Type	When Resolved	How Achieved
Compile-time (Static)	At compile time	Function Overloading, Operator Overloading
Runtime (Dynamic)	At runtime	Virtual Functions, Function Overriding

3.2 Function Overloading

Function overloading is defining multiple functions with the **same name** but **different parameters** (different number or type). The compiler selects the appropriate version at compile time.

```
int add(int a, int b) { return a+b; } float add(float a, float b) { return a+b; }
int add(int a, int b, int c) { return a+b+c; } int main() { cout << add(2,3); //
calls int version: 5 cout << add(2.5f,1.5f); // calls float version: 4.0 cout <<
add(1,2,3); // calls 3-param version: 6 }
```

3.3 Operator Overloading

Operator overloading allows standard C++ operators to work with user-defined data types (objects). The keyword **operator** is used. Cannot overload: :: .* sizeof

A) Unary Operator Overloading:

Overloads operators that work on a single operand (++ , -- , - , !).

```
class Counter { int count; public: Counter(){ count=0; } void operator++() {
count++; } // overload prefix ++ void display(){ cout << count; } }; Counter c; ++c;
// calls overloaded ++ c.display(); // 1
```

B) Binary Operator Overloading (Without Friend Function):

```
class Complex { int r, i; public: Complex(int a, int b){ r=a; i=b; } Complex
operator+(Complex c) { return Complex(r+c.r, i+c.i); } void display(){ cout << r <<
"+" << i << "i"; } }; Complex c1(2,3), c2(4,5); Complex c3 = c1+c2; c3.display(); //
6+8i
```

C) Operator Overloading Using Friend Function:

```
class Complex { int r, i; public: Complex(int a, int b){ r=a; i=b; } friend Complex
operator+(Complex c1, Complex c2); }; Complex operator+(Complex c1, Complex c2) {
return Complex(c1.r+c2.r, c1.i+c2.i); }
```

3.4 Class Template

A **template** allows writing a generic function or class that works with any data type. It provides a way of writing one code that can handle int, float, double, etc.

Syntax:

```
template class ClassName { T variable; public: void display() { cout << variable; }
};
```

Example:

```
template class Pair { T a, b; public: Pair(T x, T y){ a=x; b=y; } T larger(){ return
(a>b)?a:b; } }; Pair p1(3,7); cout << p1.larger(); // 7 Pair p2(1.5f, 2.5f); cout <<
p2.larger(); // 2.5
```

3.5 Function Template

A function template creates a generic function:

```
template <T> maximum(T a, T b) { return (a>b) ? a : b; } cout << maximum(5, 3); // 5  
cout << maximum(4.5f, 2.3f); // 4.5
```

UNIT 4: EXCEPTION HANDLING AND STREAM COMPUTATION

(Marks: 15 | Lectures: 08)

4.1 Concept of Exception Handling

An **exception** is an abnormal condition or error that occurs during program execution. Without handling, it causes the program to terminate abruptly. **Exception handling** allows the program to detect and respond to runtime errors gracefully.

Common exceptions: Division by zero, array out of bounds, file not found, null pointer access.

4.2 Types of Exceptions

Synchronous Exceptions: Occur due to program statements like division by zero, invalid array access.

Asynchronous Exceptions: Occur due to external events like hardware failure, keyboard interrupt.

Standard Exceptions: Predefined C++ exceptions: `std::bad_alloc`, `std::out_of_range`, `std::overflow_error`.

User-defined Exceptions: Custom exception classes created by the programmer using classes.

4.3 Exception Handling Techniques: try, catch, throw

C++ uses three keywords for exception handling:

try: Block of code that may throw an exception is placed here.

throw: Used to throw (raise) an exception when an error is detected.

catch: Block that catches and handles the exception thrown by try.

Syntax:

```
try { // code that may throw exception throw exception_value; } catch(data_type var)
{ // handler code }
```

Example: Division by Zero

```
int divide(int a, int b) { if(b==0) throw "Division by zero error!"; return a/b; }
int main() { try { cout << divide(10, 0); } catch(const char* msg) { cout << "Error:
" << msg; // Error: Division by zero error! } }
```

Multiple Catch Blocks:

```
try { throw 2.5; } catch(int e) { cout << "Int exception"; } catch(float e) { cout
<< "Float exception: " << e; } // Caught here catch(...) { cout << "Unknown
exception"; } // catch-all
```

4.4 Stream Computation with Console

C++ uses **streams** for input/output. A stream is a sequence of bytes flowing between a device and a program. Header: `<iostream>`

Stream Object	Purpose	Linked to
cin	Standard input stream	Keyboard
cout	Standard output stream	Screen
cerr	Standard error stream (unbuffered)	Screen
clog	Standard log stream (buffered)	Screen

```
cout << "Hello" << endl; cin >> x; cerr << "Error occurred";
```

Formatting Output with manipulators:

endl: End line and flush buffer.

setw(n): Set field width.

setprecision(n): Set decimal precision.

left / right: Left or right justify output.

4.5 Stream Computation with Files

C++ supports file I/O using the **<fstream>** library. Three classes are used:

Class	Purpose
ifstream	Read from file (input file stream)
ofstream	Write to file (output file stream)
fstream	Both read and write operations

4.6 Read and Write Operations on File

Writing to a File:

```
#include using namespace std; int main() { ofstream fout; fout.open("data.txt"); //
Create/open file fout << "Hello World" << endl; fout << "Name: Vishal";
fout.close(); // Always close file }
```

Reading from a File:

```
#include using namespace std; int main() { ifstream fin; fin.open("data.txt"); //
Open file string line; while(getline(fin, line)) { cout << line << endl; // Print
each line } fin.close(); }
```

File Operations:

open("file"): Opens a file.

close(): Closes the file.

eof(): Returns true if end of file is reached.

get() / put(): Read/write single character.

read() / write(): Read/write block of data.

seekg() / seekp(): Move read/write pointer.

tellg() / tellp(): Get current position.

SECTION 5: IMPORTANT SHORT QUESTIONS (2 MARKS)

Q1. What is OOP?

Ans: Object-Oriented Programming (OOP) is a programming paradigm based on objects that contain data and methods. It supports concepts like encapsulation, abstraction, inheritance, and polymorphism. C++ is an object-oriented language.

Q2. What is a class?

Ans: A class is a user-defined data type (blueprint) that contains data members (variables) and member functions (methods). Objects are created from a class. Keyword: class.

Q3. What is an object?

Ans: An object is an instance of a class. It represents a real-world entity with state (data members) and behavior (member functions). Example: Student s1; — s1 is an object of class Student.

Q4. What is encapsulation?

Ans: Encapsulation is the binding of data and the functions that manipulate it into a single unit (class), and hiding data from outside access. Achieved using access specifiers (private, protected, public).

Q5. What is inheritance?

Ans: Inheritance is the process by which a derived class acquires the properties and behaviors of a base class. It promotes code reusability. Keyword: class Derived : public Base { };

Q6. What is polymorphism?

Ans: Polymorphism means 'many forms'. It is the ability of a function or operator to behave differently in different contexts. Types: Compile-time (overloading) and Runtime (virtual functions).

Q7. What is a constructor?

Ans: A constructor is a special member function with the same name as the class, no return type, called automatically when an object is created. Used to initialize data members.

Q8. What is a destructor?

Ans: A destructor is a special function (same name as class, prefixed with ~) called automatically when an object goes out of scope. Used to release resources. Example: ~ClassName() { }

Q9. What is function overloading?

Ans: Function overloading allows multiple functions with the same name but different parameters (type or number) in the same scope. The correct version is selected at compile time.

Q10. What is operator overloading?

Ans: Operator overloading allows redefining the behavior of C++ operators (+, -, *, etc.) for user-defined types (objects). Done using the operator keyword.

Q11. What is a virtual function?

Ans: A virtual function is a member function declared with keyword virtual in the base class. It enables runtime polymorphism — the correct derived class version is called through a base class pointer.

Q12. What is an abstract class?

Ans: An abstract class is a class that contains at least one pure virtual function (= 0). Objects cannot be created from it. It serves as an interface for derived classes.

Q13. What is exception handling?

Ans: Exception handling is a mechanism to detect and handle runtime errors gracefully using try, catch, and throw keywords, preventing abnormal program termination.

Q14. What is throw keyword?

Ans: The throw keyword is used to raise (throw) an exception when an error is detected. Example: throw "Division by zero"; — throws a string exception.

Q15. What is a template?

Ans: A template allows writing generic functions and classes that work with any data type. Two types: Function template and Class template. Avoids code duplication.

Q16. What is a file stream?

Ans: File streams are C++ classes used for file I/O operations. ifstream — reading; ofstream — writing; fstream — both. Defined in header.

Q17. What is the difference between virtual and pure virtual function?

Ans: Virtual function has a default implementation in base class and can be overridden. Pure virtual function (= 0) has no implementation; derived class MUST override it.

Q18. What is friend class?

Ans: A friend class can access private and protected members of another class. Declared using friend class ClassName; inside the class whose members are to be accessed.

Q19. What is multiple inheritance?

Ans: Multiple inheritance is when a derived class inherits from more than one base class. Example: class C : public A, public B { }; C++ supports multiple inheritance unlike Java.

Q20. What is a copy constructor?

Ans: A copy constructor creates a new object as a copy of an existing object. Syntax: ClassName(ClassName &obj;) { }. Called when object is passed by value or returned by value.

SECTION 6: 20 MOST IMPORTANT EXAM QUESTIONS WITH ANSWERS

(Mix of 2, 4, and 6 Mark Questions – Exam Format)

Q1. [2 Marks] What is OOP? List its features.

OOP (Object-Oriented Programming) is a programming methodology that organizes software design around objects rather than logic and functions.

Features of OOP:

1. Object and Class
2. Encapsulation
3. Abstraction
4. Inheritance
5. Polymorphism
6. Message Communication

Q2. [2 Marks] Differentiate between function overloading and operator overloading.

Function Overloading: Multiple functions with same name but different parameters. Resolved at compile time.

Example: `int add(int a, int b)` and `float add(float a, float b)`.

Operator Overloading: Redefining the behavior of C++ operators (+,-,*) for user-defined objects using the operator keyword.

Example: `Complex operator+(Complex c) { ... }`

Both are examples of compile-time (static) polymorphism.

Q3. [2 Marks] What is a pure virtual function and abstract class?

Pure Virtual Function: A virtual function with no implementation in base class, assigned = 0.

Syntax: `virtual void funcName() = 0;`

Abstract Class: A class containing at least one pure virtual function. Objects of abstract class cannot be created. It forces derived classes to provide implementation.

Example: `class Shape { virtual float area() = 0; };`

Q4. [4 Marks] Explain types of constructors with examples.

A constructor is a special member function called when an object is created. Same name as class, no return type.

Types:

1. Default Constructor: No parameters.

```
class Box { Box() { width=0; } };
```

2. Parameterized Constructor: Takes arguments to initialize.

```
class Box { int w; Box(int x){ w=x; } };  
Box b(10); // calls parameterized constructor.
```

3. Copy Constructor: Creates copy of existing object.

```
Box(Box &b) { w = b.w; }  
Box b1(10); Box b2(b1); // calls copy constructor.
```

4. Constructor Overloading: Multiple constructors with different parameters.

```
Box b1; Box b2(10); Box b3(b1); // all valid.
```

Key Points: Called automatically, no return type, can be overloaded.

Q5. [4 Marks] Explain Inheritance types with examples.

Inheritance is the mechanism where a derived class acquires properties of a base class.

1. Single Inheritance: One derived from one base.

```
class Animal {}; class Dog : public Animal {};
```

2. Multiple Inheritance: One derived from multiple bases.

```
class C : public A, public B {};
```

3. Multilevel Inheritance: Chain of inheritance.

```
class A {}; class B : public A {}; class C : public B {};
```

4. Hierarchical: Multiple derived from one base.

```
class Animal {}; class Dog : public Animal {}; class Cat : public Animal {};
```

5. Hybrid: Combination of above types.

Benefit: Code reusability, extensibility, polymorphism support.

Q6. [4 Marks] Explain virtual functions with runtime polymorphism.

Virtual Function: Member function in base class declared with virtual keyword. Enables runtime polymorphism.

```
class Animal {
public:
    virtual void sound() { cout << "Generic sound"; }
};

class Dog : public Animal {
public:
    void sound() { cout << "Bark"; }
};

class Cat : public Animal {
public:
    void sound() { cout << "Meow"; }
};

int main() {
    Animal *ptr;
    Dog d; Cat c;
    ptr = &d; ptr->sound(); // Bark (runtime polymorphism)
    ptr = &c; ptr->sound(); // Meow
}
```

Without virtual: base class function would always be called.

With virtual: correct derived class function is called based on actual object.

Q7. [6 Marks] Explain Exception Handling in C++ in detail.

Definition: Exception handling is a mechanism to detect and handle runtime errors (exceptions) gracefully without crashing the program.

Keywords:

try: Encloses code that may generate exception.

throw: Throws an exception when error detected.

catch: Catches and handles the exception.

Syntax:

```
try {  
    // code that may throw  
    if(error) throw value;  
}  
catch(type var) {  
    // handle exception  
}
```

Example:

```
int divide(int a, int b) {  
    if(b == 0) throw "Division by zero!";  
    return a/b;  
}  
  
int main() {  
    try {  
        cout << divide(10, 0);  
    }  
    catch(const char* e) {  
        cout << "Error: " << e;  
    }  
}
```

Multiple Catch:

```
try { throw 100; }  
catch(int e) { cout << "Int: " << e; }  
catch(float e) { cout << "Float"; }  
catch(...) { cout << "Unknown"; } // catch-all
```

Types of Exceptions:

1. Synchronous: Division by zero, array out of bounds.
2. Asynchronous: Hardware failure, interrupt.
3. User-defined: Custom exception classes.

Re-throwing: throw; inside catch re-throws same exception.

Standard Exceptions: bad_alloc, out_of_range, overflow_error.

Conclusion: Exception handling improves program reliability and prevents abrupt termination.

Q8. [6 Marks] Explain file handling in C++ with read and write operations.

File Handling in C++ uses the <fstream> library.

Classes:

1. ofstream - Write to file.
2. ifstream - Read from file.
3. fstream - Both read and write.

Writing to File:

```
#include<fstream>  
using namespace std;
```

```

int main() {
ofstream fout;
fout.open("student.txt"); // open/create
fout << "Roll: 101" << endl;
fout << "Name: Vishal" << endl;
fout.close(); // always close
}

```

Reading from File:

```

#include<fstream>
using namespace std;
int main() {
ifstream fin;
fin.open("student.txt");
string line;
while(getline(fin, line)) {
cout << line << endl;
}
fin.close();
}

```

File Opening Modes:

ios::in - Read mode

ios::out - Write mode

ios::app - Append to file

ios::trunc - Delete and create fresh

ios::binary - Binary mode

File Pointer Functions:

seekg(n) - Move read pointer to position n

seekp(n) - Move write pointer

tellg() - Current read position

tellp() - Current write position

eof() - Check end of file

Conclusion: File handling allows permanent data storage. Always close files after operations to save data properly.

Q9. [4 Marks] Explain operator overloading with binary operator using friend function.

Operator overloading allows C++ operators to work with user-defined objects.

Using Friend Function (for binary operators):

A friend function receives both objects as parameters since it's not a member.

Example: Overloading + for Complex numbers using friend function:

```

class Complex {
int r, i;
public:
Complex(int a, int b) { r=a; i=b; }

```

```

friend Complex operator+(Complex c1, Complex c2);
    void display() { cout << r << "+" << i << "i"; }
};
Complex operator+(Complex c1, Complex c2) {
return Complex(c1.r+c2.r, c1.i+c2.i);
}
int main() {
Complex c1(2,3), c2(4,5);
Complex c3 = c1+c2;
c3.display(); // 6+8i
}

```

Advantage of friend: Can access private members of both objects.

Q10. [6 Marks] Explain Polymorphism in detail with types and examples.

Polymorphism means 'many forms'. It is the ability to present the same interface for different data types.

Types:

1. COMPILE-TIME (Static) Polymorphism:

Resolved at compile time. Achieved by:

a) Function Overloading:

```

int add(int a, int b) { return a+b; }
float add(float a, float b) { return a+b; }

```

b) Operator Overloading:

```

Complex operator+(Complex c) { return Complex(r+c.r, i+c.i); }

```

c) Function Templates:

```

template<class T> T max(T a, T b) { return a>b?a:b; }

```

2. RUNTIME (Dynamic) Polymorphism:

Resolved at runtime using virtual functions.

```

class Animal {
public: virtual void sound() { cout << "Generic"; }
};
class Dog : public Animal {
public: void sound() { cout << "Bark"; }
};
int main() {
Animal *ptr;
Dog d;
ptr = &d;
ptr->sound(); // Bark (not Generic) - runtime decision
}

```

Virtual Table (VTable): C++ uses a VTable to resolve virtual function calls at runtime.

Pure Virtual Function:

```

virtual void sound() = 0; (Abstract class)

```

Forces derived class to implement the function.

Conclusion: Polymorphism makes code flexible, extensible, and reusable. Runtime polymorphism enables designing systems that can handle new types without modifying existing code.

Q11. [2 Marks] What is encapsulation and abstraction?

Encapsulation: Binding of data and functions into a single unit (class) and restricting direct access to data using access specifiers.

Example: Private data members accessible only through public functions.

Abstraction: Hiding implementation details and showing only essential features to the user.

Example: `cout << x;` — we use it without knowing how it internally works.

Both are fundamental OOP principles implemented through classes and access specifiers.

Q12. [4 Marks] Explain class templates with example.

Template provides generic programming — write one code for all data types.

Class Template Syntax:

```
template <class T>
class ClassName {
    T data;
public:
    ClassName(T x) { data = x; }
    void display() { cout << data; }
};
```

Example:

```
template <class T>
class Pair {
    T a, b;
public:
    Pair(T x, T y) { a=x; b=y; }
    T larger() { return (a>b) ? a : b; }
};

int main() {
    Pair<int> p1(4, 7);
    cout << p1.larger(); // 7
    Pair<float> p2(1.5f, 3.2f);
    cout << p2.larger(); // 3.2
}
```

Advantage: Write once, use for any data type. No code duplication.

Function Template: `template<class T> T max(T a, T b) { return a>b?a:b; }`

Q13. [2 Marks] What is virtual base class?

A virtual base class is used in C++ to solve the Diamond Problem in multiple inheritance.

When class A is inherited by B and C, and D inherits from both B and C, without virtual, D would contain two copies of A.

With virtual inheritance: `class B : virtual public A {};` `class C : virtual public A {};`

`class D : public B, public C {};` — D gets only ONE copy of A.

This prevents ambiguity and memory waste.

Q14. [4 Marks] Explain friend function and friend class.

Friend Function:

A function that is not a member of a class but can access its private/protected members.

Declared with friend keyword inside the class, defined outside.

```
class Box {  
int width;  
public:  
    Box(int w) { width = w; }  
friend void display(Box b);  
};  
void display(Box b) {  
cout << b.width; // accesses private member  
}  
  
int main() { Box b1(10); display(b1); }
```

Friend Class:

When class B is a friend of class A, all functions of B can access private members of A.

```
class A {  
int x;  
public:  
    A(int v) { x = v; }  
friend class B; // B is friend of A  
};  
class B {  
public:  
    void show(A a) { cout << a.x; }  
};
```

Key Points:

1. Friendship is not mutual by default.
2. Friendship is not inherited.
3. Friend function is not called using dot operator.

Q15. [6 Marks] Explain Inheritance with types, visibility modes, and benefits.

Inheritance is the OOP feature where a derived class acquires properties and behaviors of a base class.

Syntax: class Derived : access_specifier Base { };

Types of Inheritance:

1. Single: class Dog : public Animal { };
2. Multiple: class C : public A, public B { };
3. Multilevel: A->B->C (chain)
4. Hierarchical: Animal->Dog, Cat, Bird
5. Hybrid: Combination

Visibility Modes:

Mode | public member | protected member | private member
public | public | protected | not inherited

protected | protected | protected | not inherited

private | private | private | not inherited

Example:

```
class Vehicle {
public:
string type;
void start() { cout << "Vehicle started"; }
};
class Car : public Vehicle {
public:
void display() {
type = "Car";
cout << type;
start(); // Inherited function
}
};
```

Benefits:

1. Code Reusability: Write once, use in derived classes.
2. Extensibility: Add new features to derived class.
3. Reduces Redundancy: Common code in base class.
4. Supports polymorphism via virtual functions.
5. Easy maintenance: Change base class, affects all derived.

Conclusion: Inheritance is a fundamental OOP concept that enables hierarchical code organization and promotes software reuse.

Q16. [2 Marks] What is message communication in OOP?

Message communication is the mechanism by which objects interact with each other by sending messages (calling member functions).

Format: object.method(parameters);

Example: student1.display(); — student1 is the object, display() is the message.

Messages are used to trigger actions and transfer data between objects in an OOP system.

Q17. [4 Marks] Explain multiple catch and re-throwing exceptions.

Multiple Catch: A try block can have multiple catch handlers for different exception types.

```
try {
int x;
cin >> x;
if(x == 0) throw 0; // int exception
if(x < 0) throw -1.5f; // float exception
if(x > 100) throw "Too large"; // string exception
}
catch(int e) { cout << "Int: " << e; }
catch(float e) { cout << "Float: " << e; }
catch(const char* e) { cout << e; }
```

```
catch(...) { cout << "Unknown exception"; } // catch all
```

Catch-all: catch(...) handles any type of exception.

Re-throwing: Use throw; (without value) inside catch to pass exception to outer handler.

```
catch(int e) {  
    cout << "Inner handler";  
    throw; // re-throw same exception  
}
```

Useful when partial handling done in inner block and full handling in outer block.

Q18. [4 Marks] What is function overloading? Write a program with 3 overloaded functions.

Function overloading allows multiple functions with same name but different signatures (parameters).

Rules:

1. Number of parameters can differ.
2. Type of parameters can differ.
3. Return type alone cannot differentiate.

Program:

```
#include<iostream>  
using namespace std;  
int area(int side) {  
    return side * side; // Square  
}  
float area(float r) {  
    return 3.14f * r * r; // Circle  
}  
int area(int l, int b) {  
    return l * b; // Rectangle  
}  
int main() {  
    cout << area(5); // 25 (Square)  
    cout << area(3.0f); // 28.26 (Circle)  
    cout << area(4, 6); // 24 (Rectangle)  
}
```

The compiler selects the correct function based on argument types at compile time.

Q19. [6 Marks] Explain OOP principles with real-world example.

OOP (Object-Oriented Programming) models real-world entities as objects with data and behavior.

Core Principles:

1. CLASS AND OBJECT:

Class = Blueprint. Object = Instance.

```
class Car { string color; void start(); };  
Car myCar; // object
```

Real world: Blueprint of a car vs actual car.

2. ENCAPSULATION:

Hide data inside class, access via methods.

```
class BankAccount {  
private: double balance;  
public: void deposit(double amt) { balance += amt; }  
};
```

Real world: ATM pin is hidden, access via interface.

3. ABSTRACTION:

Show only necessary features, hide complexity.

```
cout << x; // we don't know how it prints internally.
```

Real world: Driving car (steering/gear) without knowing engine.

4. INHERITANCE:

Child class gets features of parent.

```
class Animal { void breathe(); };  
class Dog : public Animal { void bark(); }; // Dog can breathe too
```

Real world: Dog IS-A Animal.

5. POLYMORPHISM:

Same interface, different behavior.

```
ptr->sound(); // Calls Dog::sound() or Cat::sound() based on object.
```

Real world: '+' adds numbers, concatenates strings.

6. MESSAGE COMMUNICATION:

Objects talk via method calls.

```
student.display(); // send message
```

Real world: Person talking to another person.

Advantages of OOP:

1. Modular and reusable code.
2. Real-world modeling.
3. Easy maintenance and debugging.
4. Security through encapsulation.
5. Scalable design.

Conclusion: OOP provides a structured, maintainable approach to software development.

Q20. [2 Marks] Difference between compile-time and runtime polymorphism.

Compile-time (Static) Polymorphism:

Resolved at compile time. Achieved through function overloading and operator overloading.

Fast execution. No virtual table needed.

Example: `int add(int,int)` and `float add(float,float)`.

Runtime (Dynamic) Polymorphism:

Resolved at runtime. Achieved through virtual functions and function overriding via base class pointer.

Slightly slower due to virtual table lookup.

Example: `Animal *ptr = &dog; ptr->sound();` calls Dog's version.
